첨삭 · 빨간 글씨 = 고칠 내용

FFB 휠 아두이노 펌웨어의 SRS

김도영
김창식
최민호
황찬웅

October 29, 2026

표지에 문서 버전(v1.0)과 개정 이력 표를 넣으세요. SRS는 고칠 때마다 버전을 올리고 '언제, 누가, 무엇을' 바꿨는지 남기는 문서입니다.

이 첨삭의 숫자(1ms, 100ms, 20kHz, 기본값 등)는 제안값입니다. 팀이 실험해서 정한 값으로 바꿔 넣으면 됩니다.

차 례

1. 서론 3
   1.1. 목적 3
   1.2. 범위 3
   1.3. 정의 3
   1.4. 참고 문서 3
   1.5. 문서 구성 3
2. 전체 설명 4
   2.1. 제품 관점 4
   2.2. 제품 기능 4
   2.3. 사용자 특성 4
   2.4. 제약사항 4
   2.5. 가정과 의존성 5
   2.6. 요구사항 분할 5
3. 상세 요구사항 7
   3.1. 외부 인터페이스 요구사항 7
      3.1.1. 사용자 인터페이스 7
      3.1.2. 하드웨어 인터페이스 7
      3.1.3. 소프트웨어 인터페이스 8
      3.1.4. 시리얼 통신 규칙 8
   3.2. 설계 제약 8
   3.3. 기능 요구사항
   3.4. 성능 요구사항
   3.5. 안전 요구사항
   3.6. 품질 요구사항

목차가 3.2에서 끝납니다. 1.1에서 '기능, 성능, 품질 요구사항을 정의한다'고 했고 1.5에서 'ID와 함께 나열한다'고 했는데, 그 절들이 문서에 없습니다. 위 4개 절을 새로 쓰세요(9쪽에 초안).

1. 서론

1.1. 목적

이 문서는 FFB 레이싱 휠의 Arduino Uno 펌웨어가 갖춰야 할 기능, 인터페이스, 성능, 품질 요구사항을 정의한다. 독자는 펌웨어와 게임 소프트웨어를 만드는 소프트웨어 담당, 배선을 맡는 하드웨어 담당, 시연과 검증을 맡는 팀원이다.

1.2. 범위

- 제품: FFB 휠 펌웨어
- 하는 일: 조향, 페달, 패들 입력을 읽어 시리얼 포트로 게임 소프트웨어에 보내고, 게임 소프트웨어가 보낸 힘 명령으로 모터 조작
- 포함: 센서 입력 처리, 시리얼 통신 프로토콜, 힘 계산, 모터 PWM 출력, 안전 보호, 설정과 보정, EEPROM 저장
- 제외: 게임 소프트웨어 자체, 기구와 전원 회로 설계
- 목표: 게임 소프트웨어와 시리얼로 주고받으며 모터의 힘을 안전한 범위 내에서 재현

1.3. 정의

용어	뜻
FFB	포스 피드백, 게임이 전달한 힘을 모터를 통해 핸들에 전달하는 기능
게임 소프트웨어	PC에서 실행되며 시리얼 포트로 펌웨어와 통신하는 자체 프로그램
프레임	시리얼로 주고받~~는~~ 데이터 한 덩어리. 시작 바이트, 종류, 길이, 데이터로 이루어짐 (는, 3.1.4 프레임 형식과 똑같이 맞추기)
힘 명령	게임 소프트웨어가 계산해 보내는 목표 토크. 최대 출력에 대한 비율로 표현 (숫자로: int16, -10000 ~ +10000 = -100 ~ +100 %)
대기 모드	전원을 ~~키거나~~ 켜거나 리셋한 뒤의 상태. 설정 명령을 받음
스트리밍 모드	게임 소프트웨어와 연결된 상태 (0x10을 받은 뒤 ~ 0x30 또는 통신 두절까지)

체크섬

1.4. 참고 문서

- 아두이노 우노 데이터시트 (Uno는 보드라 회로도뿐 → 'ATmega328P 데이터시트(Microchip)')
- ams AS5600 데이터시트 (+ 각 문서의 Rev. 번호와 날짜)
- Infineon BTS7960 데이터시트 (+ IEEE Std 830-1998 (SRS 작성 기준))

1.5. 문서 구성

2장은 펌웨어의 위치, 개요, 제약을 설명한다. 3장은 검증 가능한 요구사항을 ID와 함께 나열한다.

그런데 3장에 요구사항 ID가 하나도 없습니다. 여기에 ID 규칙을 적고 3장 모든 항목에 붙이세요.
FR-01… 기능 PR-01… 성능 SR-01… 안전 QR-01… 품질
각 항목에는 '1차/2차'와 '검증 방법(시험, 측정, 검사)'을 같이 적습니다.
정의 표에도 본문에 나오는 말을 추가: 틱(제어 주기 1번, 1ms) / 엔드스톱(회전 끝에서 거는 반대 힘) / 스톨(모터가 힘을 내는데 축이 안 도는 상태)

2. 전체 설명

2.1. 제품 관점

펌웨어는 FFB 휠 시스템에서 게임 소프트웨어와 하드웨어 사이에 있는 유일한 소프트웨어이다. Uno에는 USB HID 기능이 없으므로, PC와는 시리얼 포트를 열어 통신한다.

- **시스템 인터페이스**: PC와는 USB 케이블 위의 시리얼(UART)로, 하드웨어와는 I2C, ADC, 디지털 입출력, PWM으로 연결된다.
- **메모리 제약**: 플래시 32KB 중 부트로더 0.5KB를 뺀 31.5KB, SRAM 2KB, EEPROM 1KB 안에서 동작해야 한다.
- **운영**: USB를 연결하면 펌웨어가 대기 모드로 시작한다. 게임 소프트웨어가 연결 프레임을 전송하면 스트리밍 모드로 바뀐다. 모터용 24V 전원은 별도로 켜고 끈다.
- **설치 후 조정**: 회전 범위, 힘 세기, 페달 범위는 조립 후 시리얼 모니터로 보정한다.

2.2. 제품 기능

1. 조향각, 페달, 패들 입력 측정 ← 7개 기능마다 3장 요구사항 ID를 연결(FR-01…)
2. 입력 상태를 시리얼 프레임으로 게임 소프트웨어에 전송
3. 게임 소프트웨어의 힘 명령과 제어 명령 수신
4. 힘 명령과 엔드스톱을 합친 최종 토크 계산
5. 모터 드라이버 PWM 출력
6. 출력 상한, 통신 감시, 오류 정지 등 안전 보호
7. 시리얼 모니터를 통한 보정과 설정 변경

2.3. 사용자 특성

사용자	하는 일	필요한 지식
최종 사용자	핸들, 페달, 패들을 조작해 운전	필요 없음
관리자(팀원)	펌웨어 업로드, 보정, 설정 변경	아두이노, 시리얼 모니터 사용법
게임 소프트웨어 개발자(팀원)	게임 쪽 시리얼 통신 구현	시리얼 포트 프로그래밍, 바이트 단위 프레임 처리

2.4. 제약사항

- 하드웨어: Arduino Uno에서 동작해야 한다.
- 시리얼 핀: D0, D1은 USB와 연결된 유일한 시리얼 통로이므로 다른 용도로 쓸 수 없다.
- 자동 리셋: PC가 시리얼 포트를 열면 Uno가 리셋되고 부트로더를 거쳐 펌웨어가 다시 시작한다. + 리셋·부트로더 동안 D8이 떠 있음 → EN 풀다운 필요(2.5에 추가)
- 연결 감지: Uno는 USB 연결 상태를 감지할 수 없으므로, 연결 끊김은 통신이 두절된 시간으로만 판단한다.
- 수신 버퍼: 시리얼 수신 버퍼가 64B로 작으므로 매 제어 주기마다 비워야 한다.

+ 송신 버퍼도 64B입니다. 한 틱에 보낼 프레임이 버퍼 빈자리보다 크면 Serial.write()가 기다리느라 제어 주기가 밀립니다 → 0x80 전송 주기를 3.4에서 정하세요.

- 안전: 250W 모터를 구동하므로 게임 소프트웨어의 명령과 관계없이 펌웨어 안에 출력 상한이 있어야 하고 스톨을 방지해야 한다. 구체 수치는 3.5 SR-01~05에 (상한 %, 판단 조건)
- 핀 공유: I2C가 A4, A5를 쓰므로 이 핀은 페달 입력에 쓸 수 없다.
- 센서 특성: AS5600은 한 바퀴 안의 절대각만 주므로, ~~900도~~ 최대 1080도 회전에 필요한 바퀴 수는 펌웨어가 직접 세야 한다. R 명령이 1080도까지 받으므로 1080도(3바퀴) 기준으로

2.5. 가정과 의존성

- 게임 소프트웨어는 통신 규약을 따르고, 포트를 열고 ~~2초 기다린 다음~~ 통신을 시작한다. 1
- 게임 소프트웨어는 스트리밍 모드 동안 힘이 바뀌지 않아도 힘 명령을 50ms마다 한 번 이상 보낸다. 2
- 게임 소프트웨어가 노면, 코너링, 충돌 등 게임 속 힘을 계산해 하나의 힘 명령으로 보낸다.
- 조향용 AS5600은 핸들 축과 1:1로 돌고, I2C 주소 0x36을 쓴다.
- 페달용 AS5600 두 개는 OUT 핀의 아날로그 출력으로 연결한다. 페달의 유효 회전각은 18도 이상이고, 각 센서의 출력 각도 범위는 장착 전에 I2C로 하나씩 페달 행정에 맞춰 둔다. 3
- BTS7960의 RPWM, LPWM, R_EN, L_EN이 아두이노에 연결되고, 모터 전원은 SMPS 24V가 따로 공급한다. 4
- AS5600의 바퀴 수는 전원이 꺼지거나 리셋되면 사라지므로, 시작할 때 핸들은 중앙에서 반 바퀴 안에 있다고 가정한다.

2.6. 요구사항 분할

- 1차 (시연 필수): 입력 측정, 시리얼 송수신, 힘 명령 출력, 엔드스톱, 안전 보호, 설정 저장
- 2차 (이후 추가): 펌웨어 내장 스프링 및 댐퍼, 패들 조합 기능, 입력 필터 강화 5

1 'READY 줄을 받은 다음'으로. 시간을 고정하면 PC나 케이블에 따라 READY보다 먼저 보내 첫 프레임이 리셋에 먹힙니다. (최대 3초 기다려도 안 오면 연결 실패)

2 게임 쪽 약속만 있고 펌웨어 쪽 규칙이 없습니다. '스트리밍 모드에서 100ms 동안 유효 프레임이 없으면 모터를 끈다'를 3.5 SR-01로.

3 AS5600의 ZPOS/MPOS는 BURN_ANGLE로 굽지 않으면 전원을 끄면 사라집니다(굽기는 최대 3번). '맞춰서 굽는다'까지 쓰고, 굽지 않을 거면 펌웨어 P 보정만 쓴다고 적으세요.

4 + 'R_EN, L_EN에 10kΩ 풀다운 저항을 단다'. 리셋되는 동안 D8이 떠서 모터가 멋대로 켜지지 않게 합니다.

5 2차라고 했는데 3.1.4 표에는 0x21이 1차처럼 들어 있습니다. 0x21에 '(2차)' 표시. 3장 모든 요구사항에 1차/2차를 붙이면 이 절과 바로 연결됩니다.

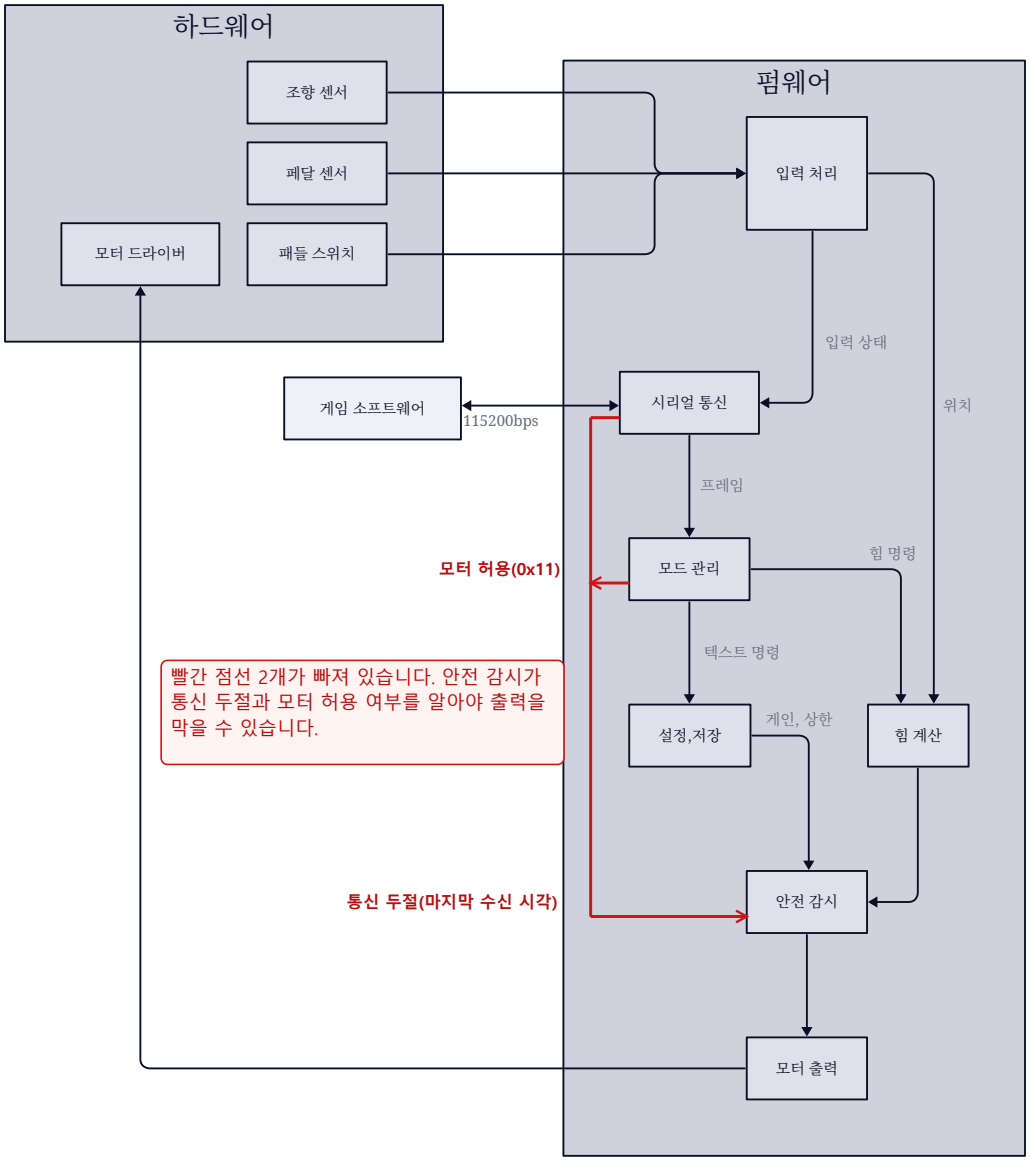


그림 1: 펌웨어 구조

입력 상태는 시리얼을 통해 게임 소프트웨어에 전달되고, 게임의 힘 명령은 모드 관리와 힘 계산을 거친 뒤 안전 감시를 통과해야만 모터로 전송된다.

+ 모드 상태도를 하나 더 넣으세요 (지금은 대기·스트리밍 전환 조건이 글에 흩어져 있음)



※ 스트리밍 → 대기로 갈 때는 항상 모터 출력 0, EN LOW

3. 상세 요구사항

3.1. 외부 인터페이스 요구사항

3.1.1. 사용자 인터페이스

- 물리 조작: 최종 사용자는 핸들, 가속 · 브레이크 페달, 패들 2개로만 펌웨어를 조작한다.
- 관리자 명령: 대기 모드에서 관리자는 시리얼 모니터로 텍스트 명령을 보낸다. 명령은 줄바꿈으로 끝나는 한 줄이고, 펌웨어는 각 명령에 `OK` 또는 `ERR`로 답해야 한다. 부팅이 끝나면 `READY` 한 줄을 출력해야 한다.

명령	기능	값 범위	기본값	
`R` ~~<%>~~ <도>	회전 범위 설정	180 – 1080	900	단위가 %가 아니라 도(°)
`C`	현재 위치를 중앙으로 설정	없음		
`G <%>`	전체 힘 게인	0 – 100	100	적용 순서를 쓰기: 힘 명령 × G → ±M%로 자름 → N
`M <%>`	최대 출력 상한	0 – 100	60	
`N <%>`	최소 PWM	0 – 100	5	힘이 0이 아닐 때만 N% 이상으로
`E <%>`	엔드스톱 세기	0 – 100	70	
`P`	페달 보정 시작	없음		절차: 끝까지 밟고 떼기 → 다시 P로 저장
`I` ~~<%>~~ <축>	축 방향 반전	0, 1, 2	0	0=조향, 1=가속, 2=브레이크. 받을 때마다 그 축 방향을 뒤집음
`V`	현재 각도, 페달, 버튼, 오류 상태 출력	없음		
`S`	설정을 EEPROM에 저장	없음		EEPROM은 S 때만 씀(수명 10만 회)
`D`	기본값 복원	없음		기본값 = 왼쪽 빨간 열

3.1.2. 하드웨어 인터페이스

신호	핀	방향	특성	
시리얼 RX, TX	D0, D1	양방향	USB 변환 칩과 연결	
조향 센서 SDA, SCL	A4, A5	양방향	I2C 400kHz, 주소 0x36	+ Wire 타임아웃(setWireTimeout)
가속 페달	A0	입력	5V 아날로그, 10비트 ADC	AS5600 OUT 기본은 VDD의 10~90% → P 보정으로 0~1023
브레이크 페달	A1	입력	5V 아날로그, 10비트 ADC	
왼쪽 패들	D4	입력	눌리면 LOW + 내부 풀업	
오른쪽 패들	~~D4~~ D5	입력	눌리면 LOW + 내부 풀업	
모터 RPWM	D9	출력	Timer1 PWM + 20kHz	
모터 LPWM	D10	출력	Timer1 PWM + 20kHz	
모터 R_EN, L_EN	D8	출력	HIGH일 때만 구동	+ 10kΩ 풀다운 (리셋 중 꺼짐)

+ 행 추가: 사용하지 않는 핀과 여유 핀(D2, D3, D5~D7, D11~D13, A2, A3)을 적어 두면 나중에 배선을 바꿀 때 충돌을 바로 압니다. D9·D10은 Timer1을 쓰므로 Servo 라이브러리와 같이 쓸 수 없다는 것도 적기.

3.1.3. 소프트웨어 인터페이스

- 게임 소프트웨어: PC의 시리얼 포트를 열어 통신한다.

3.1.4. 시리얼 통신 규칙

게임 소프트웨어와 펌웨어가 함께 지켜야 하는 규칙이다.

- 전송 설정: 115200bps, ~~8비트 데이터~~ 8N1 (데이터 8, 패리티 없음, 정지 1)
- 모드: 대기 모드에서는 텍스트 명령과 연결 프레임만 받는다. 스트리밍 모드에서는 아래 바이너리 프레임만 주고받는다.
- 프레임 형식: ~~종류, 데이터 길이, 데이터~~ 순서다.
- 값 표현: 조향각과 힘은 운전석에서 보아 오른쪽이 양수다.

시작(0xA5), 종류, 길이, 데이터, 체크섬(XOR) 순서. int16은 리틀 엔디안. 1.3 정의와 맞추기

방향	종류	이름	데이터	동작
PC → 펌웨어	0x10	연결	없음	스트리밍 모드 진입
PC → 펌웨어	0x11	모터 전원	0 또는 1 uint8	모터 출력 허용 여부
PC → 펌웨어	0x20	힘 명령	부호 있는 16비트 정수 -10000 ~ +10000	목표 토크 갱신
PC → 펌웨어	0x21	내장 힘 설정	스프링 상수, 댐퍼 상수 uint8 ×2, 0~100	내장 스프링, 댐퍼 세기 ← 2차 기능
PC → 펌웨어	0x22	중앙 설정	없음	핸들 위치를 중앙으로
PC → 펌웨어	0x30	연결 종료	없음	대기 모드 복귀
펌웨어 → PC	0x80	입력 상태	조향, 가속, 브레이크, 버튼, 상태 → 아래 ①	~~매 틱마다~~ 전송 2ms마다

- 버튼 비트: bit0 왼쪽 패들, bit1 오른쪽 패들

+ 상태 비트: bit0 모터 허용, bit1 통신 두절, bit2 센서 오류, bit3 출력 제한 중

통신 순서는 다음과 같다.

1. 게임 소프트웨어가 포트를 열고 ~~2초 기다린다~~. 그동안 아두이노가 리셋되어 `READY` 를 송신한다.
2. 0x10을 보내면 펌웨어가 0x80으로 답하기 시작한다.
3. 0x11을 보내면 모터 출력이 허용된다.
4. 끝낼 때는 0x11, 0x30 순서로 보낸 뒤 포트를 닫는다.

READY 줄을 받을 때까지 기다린다(최대 3초).

0x11 뒤에 데이터 0 (모터 끄기)

3.2. 설계 제약

- 메모리 사용: 동적 메모리 할당을 쓰지 않는다. 텍스트 메시지는 플래시에 둔다. F() 매크로
- 연산: 8비트 MCU이므로 힘 계산은 정수 또는 고정 소수점 연산을 우선한다.
- 블로킹 금지: 제어 주기 안에서 `delay()` 처럼 멈춰 있는 코드를 쓰지 않는다.
- 타이머: Timer0은 `millis()` 용으로 남겨두고, PWM은 Timer1만 쓴다. 모드 14, ICR1=799

1 0x80 데이터 8바이트: 조향 int16(0.1° 단위) · 가속 uint16 · 브레이크 uint16 · 버튼 uint8 · 상태 uint8.
프레임 = 시작 1 + 종류 1 + 길이 1 + 데이터 8 + 체크섬 1 = 12B. 115200bps(8N1)는 초당 11,520B이므로 1ms마다 보내면 12,000B/s로 넘칩니다(104%). 2ms마다 보내면 6,000B/s(52%) → '매 틱' 대신 2ms.

새로 추가할 쪽 (초안)

3.3 ~ 3.6 상세 요구사항 (예시 초안)

형식: ID [1차/2차] 요구사항 — 검증 방법. 숫자는 제안값이니 팀이 정해서 바꾸기.

3.3. 기능 요구사항

FR-01	1차	1ms마다 조향 AS5600 원시각(0~4095)을 읽고, 직전 값과 차이가 2048을 넘으면 바퀴 수를 ±1 한다. 조향각 = (바퀴 수×4096 + 원시각 - 중앙값) × 360 / 4096.	핸들을 3바퀴 돌리며 V로 각도 확인
FR-02	1차	A0, A1을 읽어 P 보정의 최소~최대를 0~1023으로 바꾼다. 범위를 벗어난 값은 끝값으로 자른다.	끝까지 밟아 0과 1023 확인
FR-03	1차	D4, D5 패들을 5ms 디바운스한 뒤 버튼 비트로 보낸다.	빠르게 연타해 두 번 눌림 없는지
FR-04	1차	|조향각|이 회전 범위의 절반을 넘으면, 넘은 각도와 E에 비례하는 반대 토크를 더한다(엔드스톱).	범위 끝에서 손을 놓아 되돌아오는지
FR-05	1차	최종 토크 = 힘 명령 × G/100 + 엔드스톱 → ±M%로 자름 → 0이 아니면 N% 이상으로 올려 PWM 출력.	고정 힘 명령을 보내며 PWM 측정
FR-06	1차	설정 구조체 + 버전 + 체크섬을 EEPROM에 쓴다. 부팅 때 체크섬이 틀리면 기본값을 쓰고 V에 표시한다.	EEPROM 1바이트를 깨고 재부팅
FR-07	2차	0x21로 받은 스프링·댐퍼 상수로 펌웨어 안에서 중앙 복원력과 감쇠를 더한다.	게임 없이 핸들을 돌려 확인

3.4. 성능 요구사항

PR-01	1차	제어 주기는 1ms(1kHz)이고, 한 주기 실행 시간은 0.8ms 이하다.	micros()로 1만 주기 측정
PR-02	1차	0x80은 2ms마다 보내고 시리얼 사용률은 60% 이하다.	PC에서 초당 프레임 수 집계
PR-03	1차	힘 명령을 받은 뒤 PWM에 반영되기까지 2ms 이하다.	핀 토글 + 오실로스코프

3.5. 안전 요구사항

SR-01	1차	스트리밍 모드에서 유효 프레임이 100ms 동안 없으면 PWM 0, EN LOW, 상태 bit1.	게임을 강제 종료
SR-02	1차	부팅 직후와 대기 모드에서는 모터 출력 0. 0x11(1)을 받은 뒤에만 출력한다.	포트를 여닫으며 확인
SR-03	1차	체크섬이나 길이가 맞지 않는 프레임은 버리고 다음 시작 바이트를 찾는다.	깨진 프레임을 일부러 전송
SR-04	1차	I2C 읽기가 3번 연속 실패하면 출력을 멈추고 상태 bit2.	조향 센서 선을 뽑기
SR-05	1차	|PWM|이 80% 이상인 상태가 2초 넘게 이어지면(스톨) 상한을 M의 50%로 낮추고 상태 bit3.	핸들을 잡고 최대 힘 명령

3.6. 품질 요구사항

QR-01	1차	플래시 사용 80% 이하, 전역 변수 SRAM 75% 이하(스택 여유).	컴파일 결과 보고
QR-02	1차	설정 변경 명령은 RAM만 바꾸고, EEPROM에는 S 명령 때만 쓴다.	코드 검사

목차(2쪽)에 3.3~3.6을 넣고, 2.2 제품 기능 7개가 각각 어느 FR에 해당하는지 표시하세요.